Roommate Ledger

Software Engineering Project Documentation

Shared Dorm Expense Splitter • React/TanStack + Python/Flask + SQLAlchemy + Supabase PostgreSQL

Presentation-ready package

This document consolidates the system requirements, architecture, database design, functional and non-functional requirements, SDLC activities, security model, testing strategy, deployment plan and principal architectural diagrams.

1. Problem Statement

Roommates frequently share groceries, electricity, water, internet, cooking gas and household supplies. Manual calculations across chats, notes and spreadsheets make it difficult to know the exact amount each roommate owes. Roommate Ledger provides a single source of truth for shared household expenses and settlements.

2. Aim and Objectives

The aim is to design and implement a reliable shared dorm expense splitter. Objectives are to record expenses, assign participants, calculate exact shares, calculate net balances, show who owes whom, record settlements, enforce household access control, provide receipts/notifications, and provide system-level administration.

3. Scope

In scope: authentication, household management, expenses, exact splitting, balances, debts, settlements, notifications, receipts, role-based access control and Super Admin oversight. Out of scope: wallet deposits/withdrawals, loans, cryptocurrency and pretending that online payment is implemented.

4. System Users

Member: records expenses, views balances and settles debts. Household Admin: manages household members and roles. Super Admin: system-wide oversight and one-time account setup.

5. Functional Requirements

FR-01 registration; FR-02 available-household selection; FR-03 invite-code joining; FR-04 authentication; FR-05 household creation; FR-06 expense creation; FR-07 exact share validation; FR-08 balance calculation; FR-09 who-owes-whom view; FR-10 partial/full settlement; FR-11 over-settlement protection; FR-12 household administration; FR-13 receipts; FR-14 notifications; FR-15 one-time Super Admin setup; FR-16 system-wide monitoring.

6. Non-Functional Requirements

Security: hashed passwords, protected sessions and server-side authorization. Accuracy: integer kobo arithmetic. Usability: plain-language balances and clear actions. Responsiveness: mobile/tablet/desktop support. Reliability: readable errors and atomic writes. Maintainability: centralized backend rules. Scalability: PostgreSQL/Supabase. Deployability: Vercel-ready frontend and PostgreSQL-compatible Flask backend.

7. Requirements Analysis

The main business rule is that the person who pays an expense is not necessarily the person who owes the full amount. Payer and participants are independent. For ■40,000 paid by Wura for four participants, each share is ■10,000 and Wura is owed ■30,000 by the other three participants.

8. Architecture

The presentation layer is React + TanStack. The application/API layer is Python Flask. SQLAlchemy provides relational persistence. The domain ledger module calculates exact splits, net balances and pairwise debts. PostgreSQL/Supabase is the production database; SQLite is the local option.

9. Database Design

Entities are User, Household, Membership, Expense, ExpenseShare, Settlement and Notification. User-to-household membership is unique. Expense shares are unique per expense/user and must sum exactly to the expense amount. Settlements are separate records and never erase the original expense.

10. Security and RBAC

The backend derives the current user from the session. Payer IDs, participant IDs, settlement recipients and roles are validated against household membership. Members cannot perform household-admin actions. Super Admin data is system-scoped. The first Super Admin setup is permanently disabled after configuration.

11. SDLC Activities

Planning → requirements engineering → analysis → architecture/design → implementation → testing → deployment → maintenance. Activities include SRS preparation, feasibility analysis, use-case modelling, database design, UI/UX design, implementation, unit/integration/system testing, security testing, deployment configuration and documentation.

12. Testing Strategy

Unit tests cover split and balance calculations. API tests cover registration, login, household joining, expense creation, settlement validation and RBAC. System tests cover complete member/admin/Super Admin journeys. Responsive tests cover mobile, tablet and desktop layouts. No test result should be reported as passed until it is actually executed.

13. Deployment

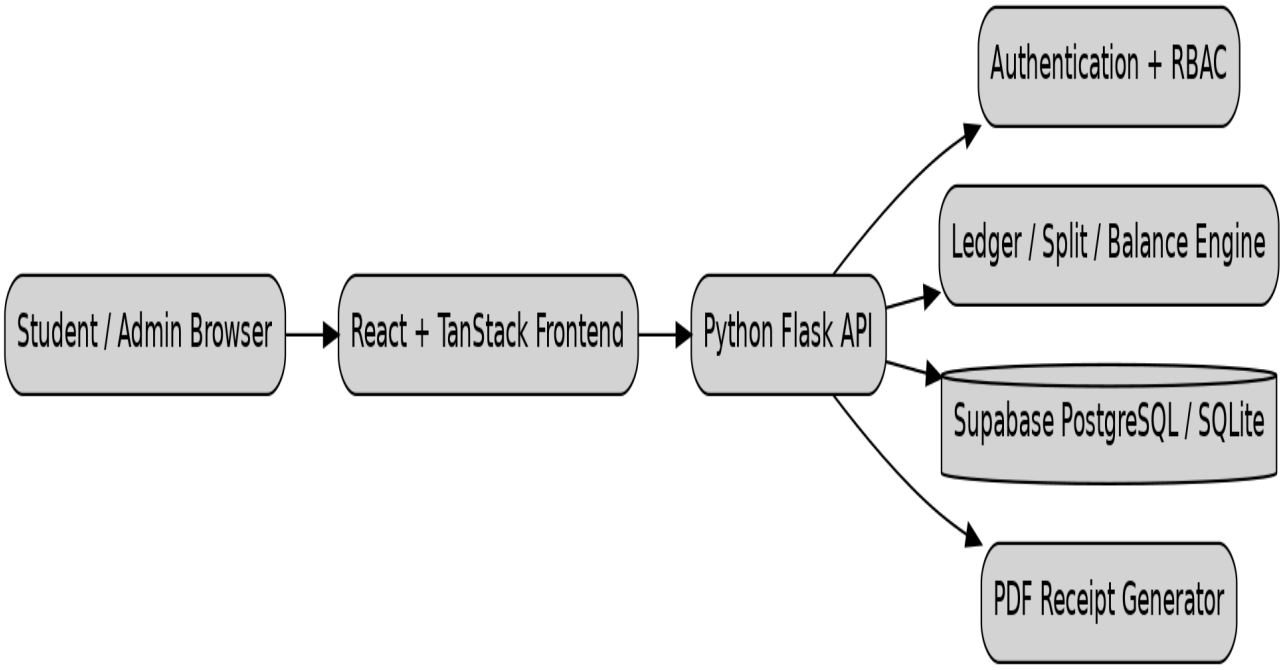
The frontend is prepared for Vercel. The backend accepts a Supabase PostgreSQL DATABASE_URL. Production requires a strong FLASK_SECRET_KEY, COOKIE_SECURE=true and the exact frontend origin in FRONTEND_ORIGIN. Local development continues to use SQLite when desired.

14. Maintenance and Future Work

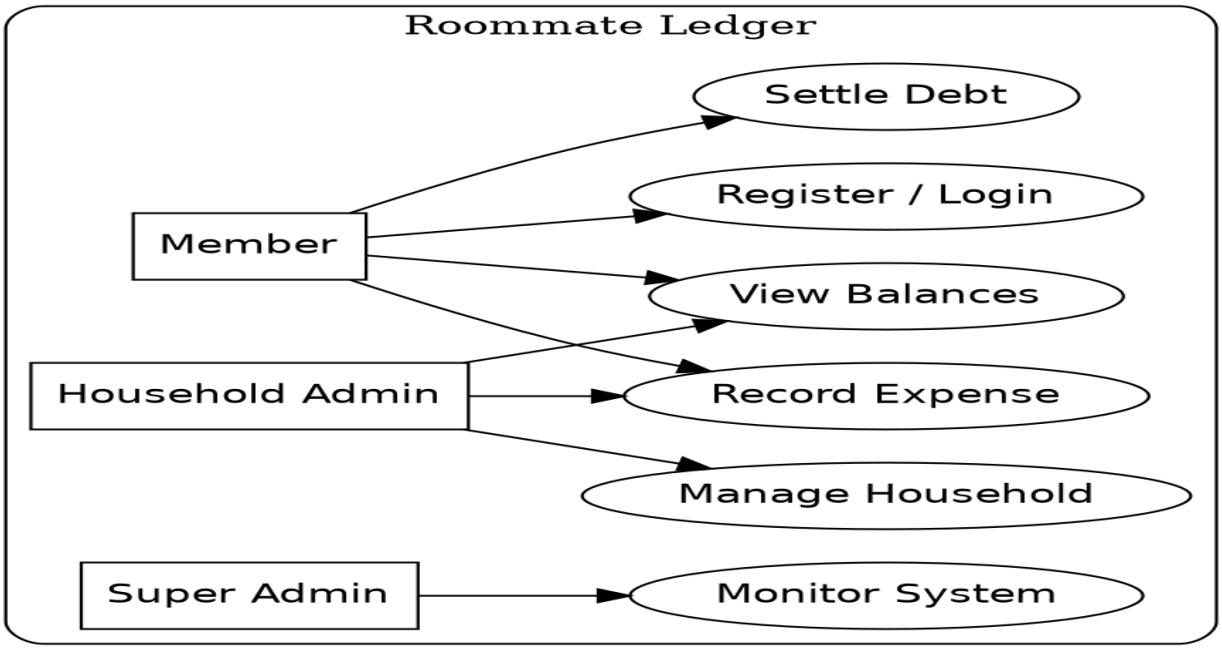
Future work may include provider-backed email delivery, online payment integration, CSV/PDF reports, advanced analytics, audit logs, multi-household membership and automated migrations. These enhancements must not weaken the core ledger rules.

15. Architectural Diagrams

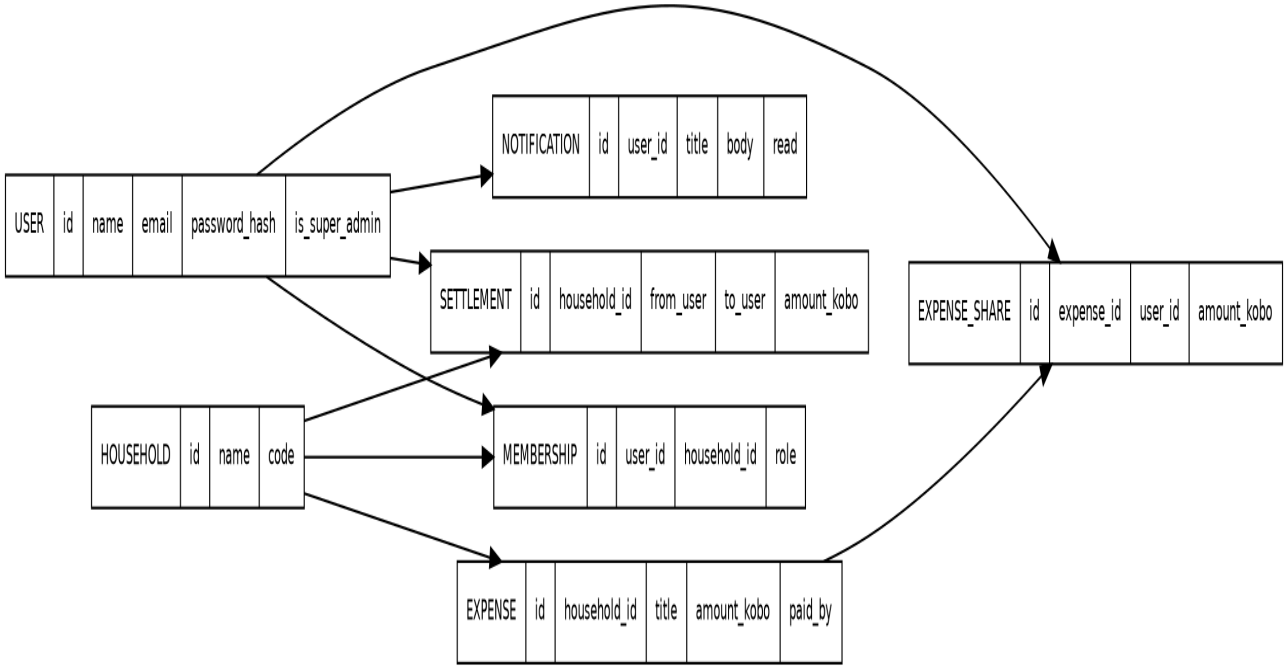
System Architecture



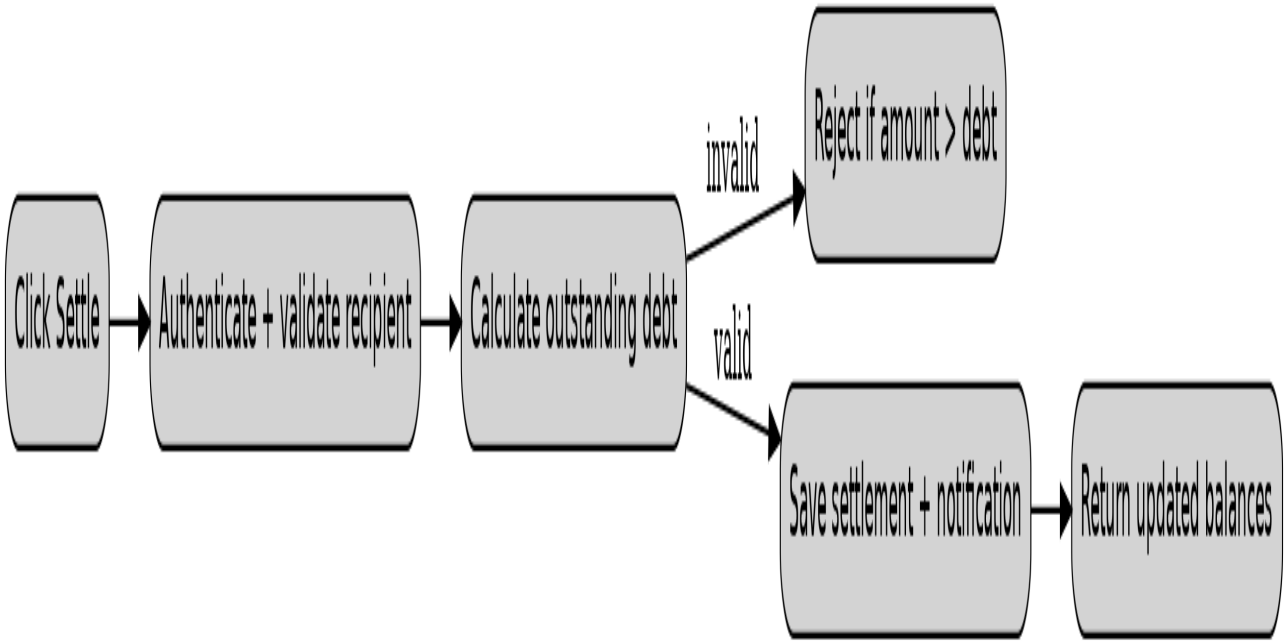
Use Case Overview



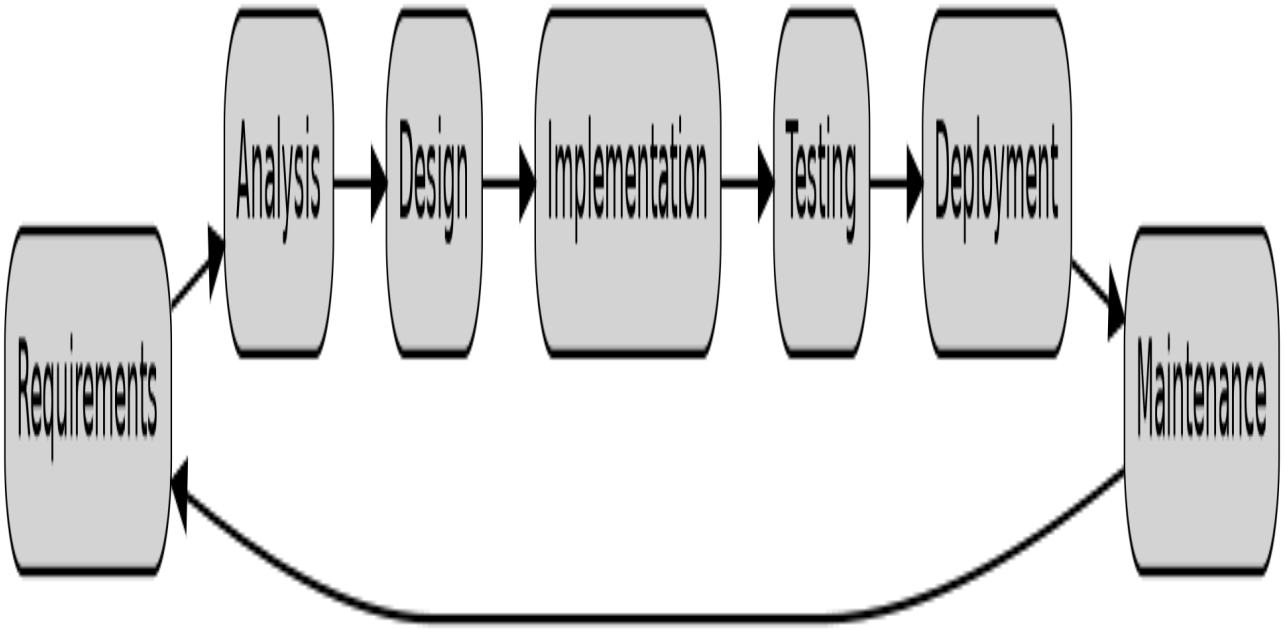
Database ERD



Settlement Flow



SDLC Lifecycle



16. Key Test Cases

ID	Scenario	Expected Result
TC-EXP-001	■40,000 / 4 participants	■10,000 each; payer net +■30,000
TC-EXP-002	■10,000 / 3 participants	■3,334 + ■3,333 + ■3,333
TC-SET-001	Full settlement	Debt becomes zero
TC-SET-002	Partial settlement	Outstanding debt decreases correctly
TC-SET-003	Over-settlement	API rejects request
TC-SA-001	First Super Admin setup	Account created
TC-SA-002	Second Super Admin setup	Setup rejected permanently
TC-RBAC-001	Member attempts admin action	403 / denied

17. Conclusion

Roommate Ledger provides a focused software solution to the shared dorm expense problem. Its design separates authentication, household management, expense splitting, balance computation and settlements while preserving a clear audit trail. The production target is Vercel for the frontend with Supabase PostgreSQL supporting the Flask backend.